

Компьютерный практикум по статистическому анализу данных

Лабораторная работа №3. Управляющие структуры

Ведьмина Александра Сергеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Циклы и условные выражения	7
3.2	Функции высшего порядка	7
3.3	Матричные операции	7
3.4	Воспроизводимость	8
4	Выполнение лабораторной работы	9
4.1	Создание проекта DrWatson	9
4.2	Примеры управляющих структур	11
4.3	Задание 1. Циклы, словарь и массив квадратов	13
4.4	Задания 2–4. Условия и функции	15
4.5	Задания 5–6. Степень матрицы и произведение	16
4.6	Задание 7. Структурные матрицы	18
4.7	Задание 8. Функция outer	19
4.8	Задание 9. Структурная система	21
4.9	Задание 10. Случайная матрица	21
4.10	Задание 11. Двойные суммы	22
4.11	Генерация производных материалов	23
4.12	Jupyter Notebook	25
4.13	Автоматические тесты	27
5	Результаты	29
6	Обсуждение результатов	30
7	Заключение	31
	Список литературы	32
A	Полный вычислительный модуль	33
B	Полный literate-сценарий	41

Список иллюстраций

4.1	Создание нового проекта средствами DrWatson	9
4.2	Установка и предварительная компиляция зависимостей	10
4.3	Состояние окружения лабораторной работы	10
4.4	Запуск Julia с окружением лабораторной работы	10
4.5	Проверка путей проекта функциями DrWatson	11
4.6	Матрица случайных цветов из палитры Colors	13
4.7	Начало вывода чисел и квадратов	14
4.8	Завершение задания 1, словарь и массив квадратов	15
4.9	Проверка чётности двумя условными конструкциями	16
4.10	Результат функции add_one	16
4.11	Матрица последовательных чисел	16
4.12	Вычисление A^3 и изменение третьего столбца	17
4.13	Построение B и вычисление $B \otimes B$	18
4.14	Матрицы Z, E и структурные матрицы Z_1-Z_4	19
4.15	Пять семейств матриц, построенных функцией outer	20
4.16	Решение структурной системы линейных уравнений	21
4.17	Матрица M и результаты запросов к её элементам	22
4.18	Точные и приближённые значения двойных сумм	23
4.19	Генерация Julia-кода, notebook и Quarto-документа	24
4.20	Рендеринг документа средствами Quarto	24
4.21	Сформированный HTML-документ лабораторной работы	25
4.22	Запуск Jupyter Lab средствами IJulia	25
4.23	Начало выполненного Jupyter Notebook	26
4.24	Матричные задания в Jupyter Notebook	26
4.25	Заключительная часть Jupyter Notebook	27
4.26	Успешное выполнение автоматических тестов	28

Список таблиц

4.1	Обобщаемые правила построения матриц	20
4.2	Итоги анализа случайной матрицы	22
5.1	Сводные результаты самостоятельной работы	29

1 Цель работы

Освоить применение циклов, условных выражений, функций и сторонних пакетов Julia для решения задач линейной алгебры и работы с матрицами.

2 Задание

Требовалось повторить примеры с циклами `while` и `for`, условными выражениями, функциями, `map`, `broadcast` и пакетом `Colors`, а затем выполнить следующие самостоятельные задания:

1. вывести числа от 1 до 100 и их квадраты, сформировать словарь и массив квадратов с помощью циклов `while` и `for`;
2. реализовать проверку чётности через `if` и тернарный оператор;
3. написать функцию `add_one`;
4. построить матрицу последовательных чисел через `map` или `broadcast`;
5. вычислить A^3 и изменить третий столбец заданной матрицы;
6. построить матрицу B размера 15×3 и вычислить $C = B^T B$;
7. получить четыре структурные матрицы размера 6×6 ;
8. реализовать обобщённую функцию `outer` и получить пять семейств матриц;
9. решить структурную систему пяти линейных уравнений;
10. исследовать воспроизводимую случайную матрицу размера 6×10 ;
11. вычислить две двойные суммы.

3 Теоретическое введение

3.1 Циклы и условные выражения

Цикл `while` выполняет тело, пока логическое условие истинно. Цикл `for` перебирает элементы диапазона или коллекции. Для выбора одной из нескольких ветвей применяются `if`, `elseif` и `else`; краткий выбор из двух значений записывается тернарным оператором `условие ? значение_1 : значение_2` [6].

3.2 Функции высшего порядка

Julia поддерживает полную, однострочную и анонимную формы определения функций. Функция `map` применяет переданную функцию к каждому элементу коллекции. `broadcast` согласует размерности аргументов и выполняет поэлементную операцию; для него предусмотрен точечный синтаксис `f.(A)` [6].

3.3 Матричные операции

Для матриц оператор `*` задаёт матричное произведение, `.*` — поэлементное, `transpose` выполняет транспонирование, а `A^n` вычисляет целую степень квадратной матрицы. Система $Ax = y$ решается оператором обратной косой черты:

```
x = A \ y
```

В работе также реализована функция `outer`. Для векторов она применяет операцию ко всем парам элементов, а для матриц заменяет умножение в обычной схеме матричного произведения:

$$C_{ij} = \sum_{k=1}^m \text{operation}(A_{ik}, B_{kj}).$$

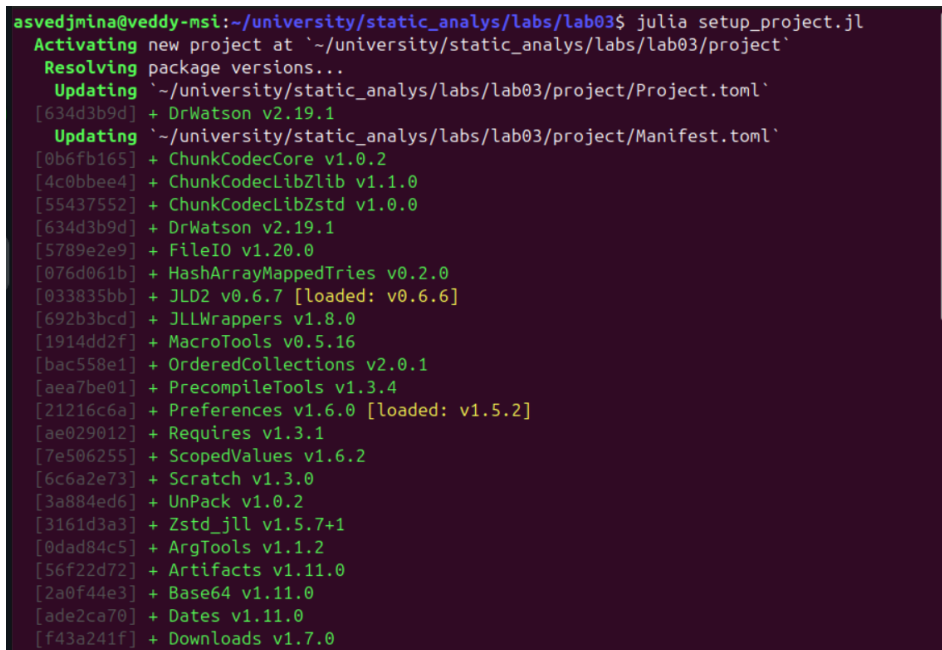
3.4 Воспроизводимость

`DrWatson` создаёт стандартную структуру вычислительного проекта и предоставляет функции независимого от текущего каталога доступа к исходникам, данным и результатам [1]. `Literate` формирует из одного комментированного Julia-файла обычный сценарий, `notebook` и `Quarto`-документ [2; 5]. Пакет `IJulia` обеспечивает работу Julia в Jupyter [4].

4 Выполнение лабораторной работы

4.1 Создание проекта DrWatson

Из каталога lab03 был запущен сценарий `setup_project.jl`. Функция `initialize_project` создала новый проект `Lab03ControlStructures`, стандартные каталоги и окружение с пакетом `DrWatson` (Рисунок 4.1).



```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia setup_project.jl
Activating new project at '~/.university/static_analys/labs/lab03/project'
Resolving package versions...
Updating '~/.university/static_analys/labs/lab03/project/Project.toml'
[634d3b9d] + DrWatson v2.19.1
Updating '~/.university/static_analys/labs/lab03/project/Manifest.toml'
[0b6fb165] + ChunkCodecCore v1.0.2
[4c0bbe4] + ChunkCodecLibZlib v1.1.0
[55437552] + ChunkCodecLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FileIO v1.20.0
[076d061b] + HashArrayMappedTries v0.2.0
[033835bb] + JLD2 v0.6.7 [loaded: v0.6.6]
[692b3bcd] + JLLWrappers v1.8.0
[1914dd2f] + MacroTools v0.5.16
[bac558e1] + OrderedCollections v2.0.1
[aea7be01] + PrecompileTools v1.3.4
[21216c6a] + Preferences v1.6.0 [loaded: v1.5.2]
[ae029012] + Requires v1.3.1
[7e506255] + ScopedValues v1.6.2
[6c6a2e73] + Scratch v1.3.0
[3a884ed6] + UnPack v1.0.2
[3161d3a3] + Zstd_jll v1.5.7+1
[0dad84c5] + ArgTools v1.1.2
[56f22d72] + Artifacts v1.11.0
[2a0f44e3] + Base64 v1.11.0
[ade2ca70] + Dates v1.11.0
[f43a241f] + Downloads v1.7.0
```

Рисунок 4.1: Создание нового проекта средствами DrWatson

После активации окружения установлены `Colors`, `DrWatson`, `IJulia`, `Literate`,

LinearAlgebra и Random (Рисунок 4.2). Команда `Pkg.status()` подтвердила состав окружения (Рисунок 4.3).

```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project proje
ct/add_packages.jl
  Activating project at `~/university/static_analys/labs/lab03/project`
Зависимости лабораторной работы № 3 установлены.
Project Lab03ControlStructures v0.1.0
Status `~/university/static_analys/labs/lab03/project/Project.toml`
 [5ae59095] Colors v0.13.2
 [634d3b9d] DrWatson v2.19.1
 [7073ff75] IJulia v1.34.4
 [98b081ad] Literate v2.21.0
 [37e2e46d] LinearAlgebra v1.12.0
 [9a3f8284] Random v1.11.0
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$
```

Рисунок 4.2: Установка и предварительная компиляция зависимостей

```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project -e 'using Pkg; Pkg.status()'
Project Lab03ControlStructures v0.1.0
Status `~/university/static_analys/labs/lab03/project/Project.toml`
 [5ae59095] Colors v0.13.2
 [634d3b9d] DrWatson v2.19.1
 [7073ff75] IJulia v1.34.4
 [98b081ad] Literate v2.21.0
 [37e2e46d] LinearAlgebra v1.12.0
 [9a3f8284] Random v1.11.0
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$
```

Рисунок 4.3: Состояние окружения лабораторной работы

Julia была запущена с параметром `--project`, после чего `@quickactivate` активировал проект по имени (Рисунок 4.4, Рисунок 4.5).

```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=/home/asvedjmina/university/static_analys/labs/lab03/project
```

```
(_) | Documentation: https://docs.julia.org  
(_)|  
|_| | Type "?" for help, "]?" for Pkg help.  
|_| |  
|_| | Version 1.12.5 (2026-02-09)  
|_| | Official https://julia.org release  
|_| |  
  
julia>
```

Рисунок 4.4: Запуск Julia с окружением лабораторной работы

```
asvedjmina@veddy-msi: ~/university/static_analys/labs/lab03
julia> using DrWatson
^@
julia> cd("/home/asvedjmina/university/static_analys/labs/lab03/project")

julia> @quickactivate "Lab03ControlStructures"

julia> pwd()
"/home/asvedjmina/university/static_analys/labs/lab03/project"

julia> Base.active_project()
"/home/asvedjmina/university/static_analys/labs/lab03/project/Project.toml"

julia> projectname()
"Lab03ControlStructures"

julia> projectdir()
"/home/asvedjmina/university/static_analys/labs/lab03/project"

julia> srcdir()
"/home/asvedjmina/university/static_analys/labs/lab03/project/src"

julia> scriptsdir()
"/home/asvedjmina/university/static_analys/labs/lab03/project/scripts"

julia> datadir()
"/home/asvedjmina/university/static_analys/labs/lab03/project/data"

julia>
```

Рисунок 4.5: Проверка путей проекта функциями DrWatson

4.2 Примеры управляющих структур

Цикл `while` использован для вывода чисел от 1 до 10 и перебора массива имён. Те же имена и нечётные числа диапазона `1:2:10` выведены циклом `for`:

```
n = 0
while n < 10
    global n += 1
    println(n)
end

for friend in myfriends
    println("Hi $friend, it's great to see you!")
end

for number in 1:2:10
```

```
println(number)
end
```

Три эквивалентных способа построения матрицы проверены оператором равенства:

```
A_loop = fill{0, (m, n)}
for row in 1:m, column in 1:n
    A_loop[row, column] = row + column
end

B_loop = [row + column for row in 1:m,
          column in 1:n]
@assert A_loop == B_loop
```

Условный пример FizzBuzz реализован последовательностью if-elseif-else. Показаны полная, однострочная и анонимная формы функций. Для матрицы

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

сопоставлены $f(A) = A^2$ и поэлементное $f.(A)$. Пакет Colors использован для воспроизводимого построения матрицы цветов из палитры ста различных оттенков [3]:

```
palette = distinguishable_colors(100)
color_rng = MersenneTwister(2026)
color_matrix = rand(color_rng, palette, 3, 3)
display(color_matrix)
```

Полученная матрица показана на Рисунок 4.6. Это фактический графический вывод ячейки notebook для фиксированного начального состояния генератора.



Рисунок 4.6: Матрица случайных цветов из палитры Colors

4.3 Задание 1. Циклы, словарь и массив квадратов

Реализации `squares_with_while` и `squares_with_for` создают пары (число, квадрат), словарь `squares` и массив `squares_arr`. Ниже приведена реализация через `while`; версия с `for` отличается управляющим циклом.

```
function squares_with_while(; limit::Integer=100)
    pairs = Tuple{Int, Int}[]
    squares = Dict{Int, Int}()
    squares_arr = Int[]
    number = 1

    while number <= limit
        square = number^2
        push!(pairs, (number, square))
        squares[number] = square
    end
end
```

```

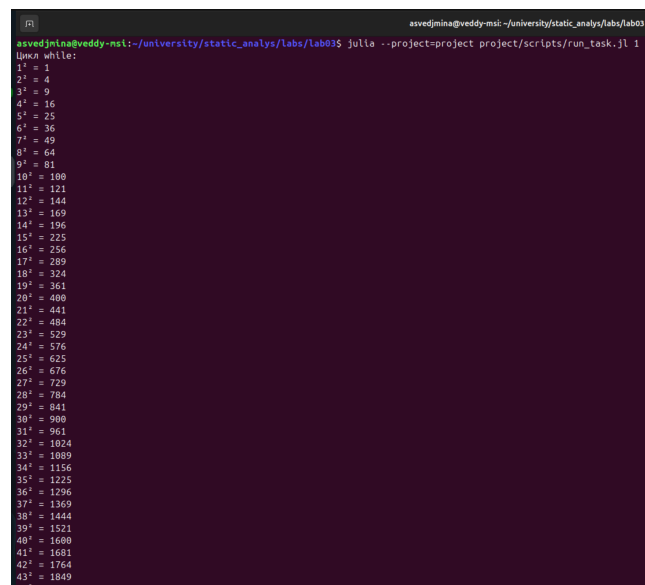
    push!(squares_arr, square)

    number += 1
end

return (numbers_and_squares=pairs,
        squares=squares,
        squares_arr=squares_arr)
end

```

Первая часть вывода чисел и квадратов представлена на Рисунок 4.7, а окончание диапазона, словарь и массив — на Рисунок 4.8.



```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03 julia --project=project project/scripts/run_task.jl 1
while:
1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25
6^2 = 36
7^2 = 49
8^2 = 64
9^2 = 81
10^2 = 100
11^2 = 121
12^2 = 144
13^2 = 169
14^2 = 196
15^2 = 225
16^2 = 256
17^2 = 289
18^2 = 324
19^2 = 361
20^2 = 400
21^2 = 441
22^2 = 484
23^2 = 529
24^2 = 576
25^2 = 625
26^2 = 676
27^2 = 729
28^2 = 784
29^2 = 841
30^2 = 900
31^2 = 961
32^2 = 1024
33^2 = 1089
34^2 = 1156
35^2 = 1225
36^2 = 1296
37^2 = 1369
38^2 = 1444
39^2 = 1521
40^2 = 1600
41^2 = 1681
42^2 = 1764
43^2 = 1849
44^2 = 1936

```

Рисунок 4.7: Начало вывода чисел и квадратов

```

77² = 5929
78² = 6084
79² = 6241
80² = 6400
81² = 6561
82² = 6724
83² = 6889
84² = 7056
85² = 7225
86² = 7396
87² = 7569
88² = 7744
89² = 7921
90² = 8100
91² = 8281
92² = 8464
93² = 8649
94² = 8836
95² = 9025
96² = 9216
97² = 9409
98² = 9604
99² = 9801
100² = 10000
squares = Dict{5 => 25, 56 => 3136, 35 => 1225, 55 => 3025, 60 => 3600, 30 => 900, 32 => 1024, 6 => 36, 67 => 4489, 45 => 2025, 73 => 5329, 64 => 4096, 90 => 8100, 4 => 16, 13 => 169,
54 => 2916, 63 => 3969, 86 => 7396, 91 => 8281, 62 => 3844, 58 => 3364, 52 => 2704, 12 => 144, 28 => 784, 75 => 5625, 23 => 529, 92 => 8464, 41 => 1681, 43 => 1849, 11 => 121, 36 => 12
96, 68 => 4624, 69 => 4761, 98 => 9604, 82 => 6724, 85 => 7225, 39 => 1521, 84 => 7056, 77 => 5929, 7 => 49, 25 => 625, 95 => 9025, 71 => 5041, 66 => 4356, 76 => 5776, 34 => 1156, 50 =>
> 2500, 59 => 3481, 93 => 8649, 2 => 4, 10 => 100, 18 => 324, 26 => 676, 27 => 729, 42 => 1764, 87 => 7569, 100 => 10000, 79 => 6241, 16 => 256, 20 => 400, 81 => 6561, 19 => 361, 49 =>
2401, 44 => 1936, 9 => 81, 31 => 961, 74 => 5476, 61 => 3721, 29 => 841, 94 => 8836, 46 => 2116, 57 => 3249, 70 => 4900, 21 => 441, 38 => 1444, 88 => 7744, 78 => 6084, 72 => 5184, 24
=> 576, 8 => 64, 17 => 289, 37 => 1369, 1 => 1, 53 => 2809, 22 => 484, 47 => 2209, 83 => 6889, 99 => 9801, 89 => 7921, 14 => 196, 3 => 9, 80 => 6400, 96 => 9216, 51 => 2601, 33 => 1089
, 48 => 2304, 40 => 1600, 48 => 2304, 15 => 225, 65 => 4225, 97 => 9409)
squares_arr = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1156, 1225, 12
96, 1369, 1444, 1521, 1600, 1681, 1764, 1849, 1936, 2025, 2116, 2209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025, 3136, 3249, 3364, 3481, 3600, 3721, 3844, 3969, 4096, 4225, 4356,
4489, 4624, 4761, 4900, 5041, 5184, 5329, 5476, 5625, 5776, 5929, 6084, 6241, 6400, 6561, 6724, 6889, 7056, 7225, 7396, 7569, 7744, 7921, 8100, 8281, 8464, 8649, 8836, 9025, 9216, 9409
, 9604, 9801, 10000]
savedjina@veddy-nsl:~/university/static_analysis/labs/lab03$

```

Рисунок 4.8: Завершение задания 1, словарь и массив квадратов

4.4 Задания 2–4. Условия и функции

Проверка чётности выполнена двумя способами:

```

function even_or_odd(number::Integer)

    if iseven(number)

        return number

    else

        return "нечётное"

    end

end

even_or_odd_ternary(number::Integer) =

    iseven(number) ? number : "нечётное"

```

Обе реализации дали одинаковые результаты (Рисунок 4.9). Функция `add_one(value) = value + one(value)` увеличила 41 до 42 (Рисунок 4.10).

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 2
1 → нечётное / нечётное
2 → 2 / 2
3 → нечётное / нечётное
4 → 4 / 4
5 → нечётное / нечётное
6 → 6 / 6
7 → нечётное / нечётное
8 → 8 / 8
9 → нечётное / нечётное
10 → 10 / 10
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

Рисунок 4.9: Проверка чётности двумя условными конструкциями

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 3
add_one(41) = 42
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

Рисунок 4.10: Результат функции add_one

Матрица последовательных чисел построена применением `map(add_one, ...)`:

```

function increasing_matrix(rows=3, columns=4)
    predecessors = [
        (row - 1) * columns + column - 1
        for row in 1:rows, column in 1:columns
    ]
    return map(add_one, predecessors)
end

```

Получена матрица размера 3×4 с числами от 1 до 12 (Рисунок 4.11).

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 4
3×4 Matrix{Int64}:
 1  2  3  4
 5  6  7  8
 9 10 11 12
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

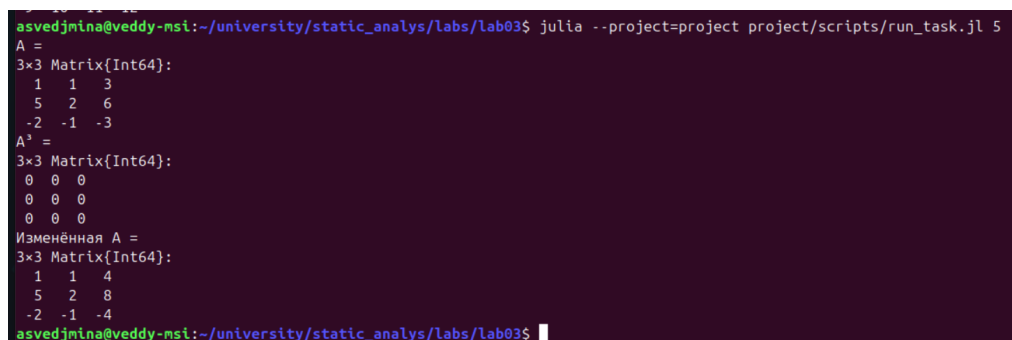
Рисунок 4.11: Матрица последовательных чисел

4.5 Задания 5–6. Степень матрицы и произведение

Для заданной матрицы вычислена третья степень и изменён третий столбец:

```
A = [1 1 3; 5 2 6; -2 -1 -3]
A_cubed = A^3
A_modified = copy(A)
A_modified[:, 3] .= A[:, 2] .+ A[:, 3]
```

Матрица оказалась нильпотентной третьей степени: $A^3 = 0$. Изменённая матрица и промежуточные результаты приведены на Рисунок 4.12.



```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 5
A =
3×3 Matrix{Int64}:
 1  1  3
 5  2  6
-2 -1 -3
A³ =
3×3 Matrix{Int64}:
 0  0  0
 0  0  0
 0  0  0
Изменённая A =
3×3 Matrix{Int64}:
 1  1  4
 5  2  8
-2 -1 -4
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$
```

Рисунок 4.12: Вычисление A^3 и изменение третьего столбца

Матрица B сформирована повторением строки $(10, -10, 10)$:

```
B = repeat([10 -10 10], 15, 1)
C = transpose(B) * B
```

В результате

$$C = \begin{pmatrix} 1500 & -1500 & 1500 \\ -1500 & 1500 & -1500 \\ 1500 & -1500 & 1500 \end{pmatrix}.$$

Вывод матриц B и C представлен на Рисунок 4.13.

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 6
B =
15×3 Matrix{Int64}:
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
BᵀB =
3×3 Matrix{Int64}:
 1500  -1500  1500
 -1500   1500 -1500
 1500  -1500  1500
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

Рисунок 4.13: Построение B и вычисление $B^T B$

4.6 Задание 7. Структурные матрицы

Матрицы построены одним двойным циклом по строкам и столбцам. Их элементы определены через расстояние до диагонали и чётность суммы индексов:

```

for row in 1:n, column in 1:n
    Z1[row, column] = abs(row - column) == 1
    Z2[row, column] = abs(row - column) in (0, 2)
    reflected = n - column + 1
    Z3[row, column] = abs(row - reflected) in (0, 2)
    Z4[row, column] = iseven(row + column)
end

```

Такое описание автоматически обобщается на любой положительный размер n . Нулевая, единичная и четыре требуемые матрицы показаны на Рисунок 4.14.

```

asvedjmina@veddy-nsi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 7
\Z =
6x6 Matrix{Int64}:
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
E =
6x6 Matrix{Int64}:
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
Z1 =
6x6 Matrix{Int64}:
 0  1  0  0  0  0
 1  0  1  0  0  0
 0  1  0  1  0  0
 0  0  1  0  1  0
 0  0  0  1  0  1
 0  0  0  0  1  0
Z2 =
6x6 Matrix{Int64}:
 1  0  1  0  0  0
 0  1  0  1  0  0
 1  0  1  0  1  0
 0  1  0  1  0  1
 0  0  1  0  1  0
 0  0  0  1  0  1
Z3 =
6x6 Matrix{Int64}:
 0  0  0  1  0  1
 0  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  0
 1  0  1  0  0  0
Z4 =
6x6 Matrix{Int64}:
 1  0  1  0  0  0
 0  1  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0

```

Рисунок 4.14: Матрицы Z, E и структурные матрицы Z₁–Z₄

4.7 Задание 8. Функция outer

Для векторов использовано попарное применение операции, для матриц — свёртка по общему индексу:

```

function outer(x::AbstractVector,
              y::AbstractVector,
              operation::Function)
    return [operation(a, b) for a in x, b in y]
end

function outer(A::AbstractMatrix,
              B::AbstractMatrix,
              operation::Function)
    size(A, 2) == size(B, 1) ||
        throw(DimensionMismatch("Несогласованные размеры"))

```

```

return [
    sum(operation(A[i, k], B[k, j]))
    for k in axes(A, 2))
    for i in axes(A, 1), j in axes(B, 2)
]
end

```

Проверено равенство `outer(A, B, *) == A * B`. Пять требуемых матриц получены операциями сложения, возведения в степень и взятия остатка (Рисунок 4.15).

```

asvedj@nina@veddy-mst:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 8
A1 =
5×5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  5
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
A2 =
5×5 Matrix{Int64}:
 0  0  0  0  0
 1  1  1  1  1
 2  4  8  16  32
 3  9  27  81  243
 4  16  64  256  1024
A3 =
5×5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  0
 2  3  4  0  1
 3  4  0  1  2
 4  0  1  2  3
A4 =
10×10 Matrix{Int64}:
 0  1  2  3  4  5  6  7  8  9
 1  2  3  4  5  6  7  8  9  0
 2  3  4  5  6  7  8  9  0  1
 3  4  5  6  7  8  9  0  1  2
 4  5  6  7  8  9  0  1  2  3
 5  6  7  8  9  0  1  2  3  4
 6  7  8  9  0  1  2  3  4  5
 7  8  9  0  1  2  3  4  5  6
 8  9  0  1  2  3  4  5  6  7
 9  0  1  2  3  4  5  6  7  8
A5 =
9×9 Matrix{Int64}:
 0  8  7  6  5  4  3  2  1
 1  0  8  7  6  5  4  3  2

```

Рисунок 4.15: Пять семейств матриц, построенных функцией `outer`

Таблица 4.1: Обобщаемые правила построения матриц

Матрица	Размер	Формула элемента
A_1	5×5	$i + j$
A_2	5×5	i^{j+1}
A_3	5×5	$(i + j) \bmod 5$

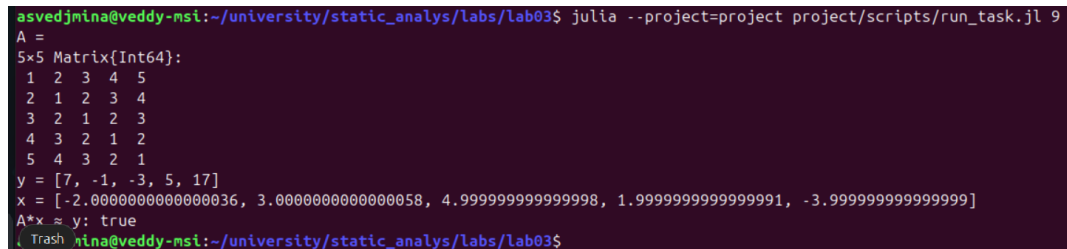
Матрица	Размер	Формула элемента
A_4	10×10	$(i + j) \bmod 10$
A_5	9×9	$(i - j) \bmod 9$

4.8 Задание 9. Структурная система

Коэффициенты системы описываются единой формулой $A_{ij} = |i - j| + 1$. Поэтому метод непосредственно обобщается на произвольное число уравнений:

```
n = length(y)
A = [abs(i - j) + 1 for i in 1:n, j in 1:n]
x = A \ y
```

Получено решение $x = (-2, 3, 5, 2, -4)^T$; проверка $A * x \approx y$ вернула true (Рисунок 4.16).



```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 9
A =
5x5 Matrix{Int64}:
 1  2  3  4  5
 2  1  2  3  4
 3  2  1  2  3
 4  3  2  1  2
 5  4  3  2  1
y = [7, -1, -3, 5, 17]
x = [-2.0000000000000036, 3.0000000000000058, 4.999999999999998, 1.9999999999999991, -3.999999999999999]
A*x ≈ y: true
Trash mina@veddy-msi:~/university/static_analys/labs/lab03$
```

Рисунок 4.16: Решение структурной системы линейных уравнений

4.9 Задание 10. Случайная матрица

Генератор MersenneTwister(2026) обеспечивает повторяемость матрицы:

```
rng = MersenneTwister(2026)
M = rand(rng, 1:10, 6, 10)
```

```
greater_counts = vec(sum(M .> 4; dims=2))
matching_rows = findall(
    vec(sum(M .== 7; dims=2)) .== 2,
)
column_sums = vec(sum(M; dims=1))
```

Результаты обработки показаны на Рисунок 4.17 и сведены в Таблица 4.2.

```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 10
M =
6x10 Matrix{Int64}:
 9  1  1  4  6  4  7  4 10  3
 2  6  2  5  1 10  4  2  7 10
10  6 10 10  3 10  8 10 10  1
 4  3  6  6  7  4  8  5  1  5
 8  5  6  2  2  3  2  2  4  6
 8  2  2  2 10  3  7  8  7  2
Количество > 4: [4, 5, 8, 6, 4, 5]
Строки с двумя числами 7: [6]
Суммы столбцов: [41, 23, 27, 29, 29, 34, 36, 31, 39, 27]
Пары столбцов с общей суммой > 75: [(columns = (1, 7), sum = 77), (columns = (1, 9), sum = 80)]
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$
```

Рисунок 4.17: Матрица M и результаты запросов к её элементам

Таблица 4.2: Итоги анализа случайной матрицы

Показатель	Результат
Число элементов > 4 по строкам	[4, 5, 8, 6, 4, 5]
Строки с двумя значениями 7	[6]
Суммы столбцов	[41, 23, 27, 29, 29, 34, 36, 31, 39, 27]
Пары с общей суммой > 75	(1, 7): 77, (1, 9): 80

4.10 Задание 11. Двойные суммы

Суммы вычислены точно с помощью рациональных чисел `BigInt`, что исключило переполнение при приведении дробей к общему знаменателю:

```

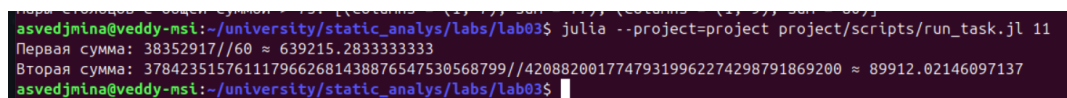
first_sum = sum(
    big(i)^4 // big(3 + j)
    for i in 1:20 for j in 1:5
)
second_sum = sum(
    big(i)^4 // big(3 + i * j)
    for i in 1:20 for j in 1:5
)

```

Получены значения

$$S_1 = \frac{38352917}{60} \approx 639215,2833, \quad S_2 \approx 89912,02146.$$

Точный вывод обеих сумм приведён на Рисунок 4.18.



```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/run_task.jl 11
Первая сумма: 38352917//60 ≈ 639215.2833333333
Вторая сумма: 3784235157611179662681438876547530568799//42088200177479319962274298791869200 ≈ 89912.02146097137
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

Рисунок 4.18: Точные и приближённые значения двойных сумм

4.11 Генерация производных материалов

Сценарий `generate_outputs.jl` создал чистый Julia-файл, выполненный notebook, исходник Quarto и HTML-документ (Рисунок 4.19).

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ julia --project=project project/scripts/generate_outputs.jl
[ Info: generating plain script file from '~/university/static_analys/labs/lab03/project/scripts/lab03_literate.jl'
[ Info: writing result to '~/university/static_analys/labs/lab03/project/generated/lab03_code.jl'
[ Info: generating notebook from '~/university/static_analys/labs/lab03/project/scripts/lab03_literate.jl'
[ Info: executing notebook 'lab03_control_structures.ipynb'
[ Info: writing result to '~/university/static_analys/labs/lab03/project/generated/lab03_control_structures.ipynb'
[ Info: generating markdown page from '~/university/static_analys/labs/lab03/project/scripts/lab03_literate.jl'
[ Info: writing result to '~/university/static_analys/labs/lab03/project/generated/lab03_quarto.qmd'
pandoc
to: html
output-file: lab03_quarto.html
standalone: true
section-divs: true
html-math-method: mathjax
wrap: none
default-image-extension: png

metadata
document-css: false
link-citations: true
date-format: long
lang: en
title: Лабораторная работа № 3. Управляющие структуры

Output created: lab03_quarto.html

Сгенерирован чистый код: /home/asvedjmina/university/static_analys/labs/lab03/project/generated/lab03_code.jl
Сгенерирован и выполнен notebook: /home/asvedjmina/university/static_analys/labs/lab03/project/generated/lab03_control_structures.ipynb
Сгенерирован Quarto-документ: /home/asvedjmina/university/static_analys/labs/lab03/project/generated/lab03_quarto.qmd
Сгенерирован HTML: /home/asvedjmina/university/static_analys/labs/lab03/project/generated/lab03_quarto.html
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

Рисунок 4.19: Генерация Julia-кода, notebook и Quarto-документа

Повторный рендеринг Quarto успешно сформировал HTML (Рисунок 4.20), который был проверен в браузере (Рисунок 4.21).

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$ quarto render project/generated/lab03_quarto.qmd --to html
pandoc
to: html
output-file: lab03_quarto.html
standalone: true
section-divs: true
html-math-method: mathjax
wrap: none
default-image-extension: png

metadata
document-css: false
link-citations: true
date-format: long
lang: en
title: Лабораторная работа № 3. Управляющие структуры

Output created: lab03_quarto.html
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$

```

Рисунок 4.20: Рендеринг документа средствами Quarto

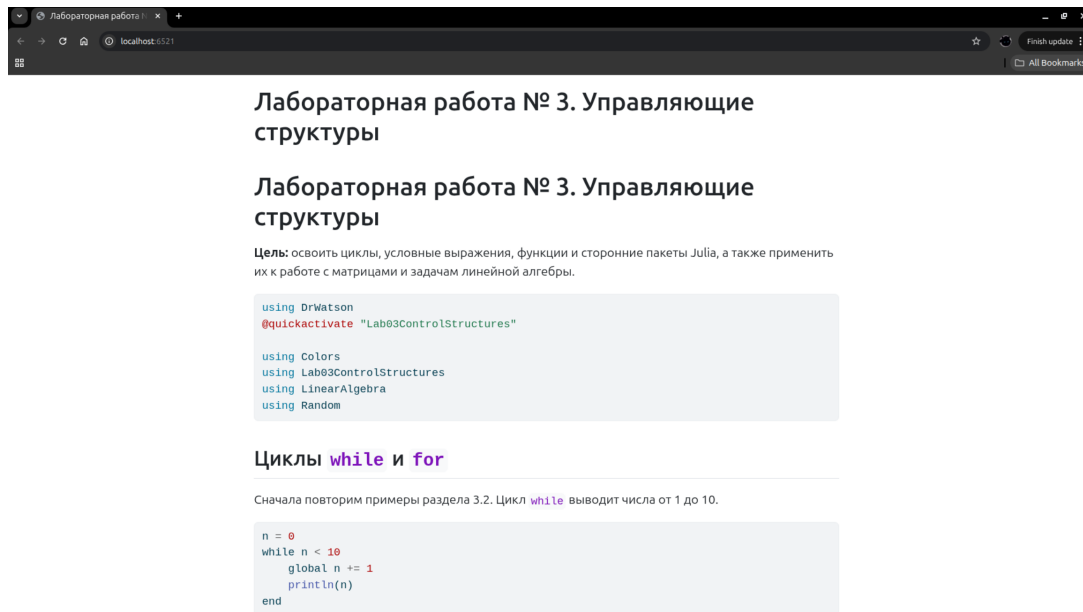


Рисунок 4.21: Сформированный HTML-документ лабораторной работы

4.12 Jupyter Notebook

Jupyter Lab запущен из окружения проекта (Рисунок 4.22). Выполненный notebook содержит теоретические примеры, самостоятельные задания и выводы.



Рисунок 4.22: Запуск Jupyter Lab средствами Julia

Начало notebook с примерами циклов показано на Рисунок 4.23, матричные

задания — на Рисунок 4.24, заключительная часть с суммами и выводом — на Рисунок 4.25.

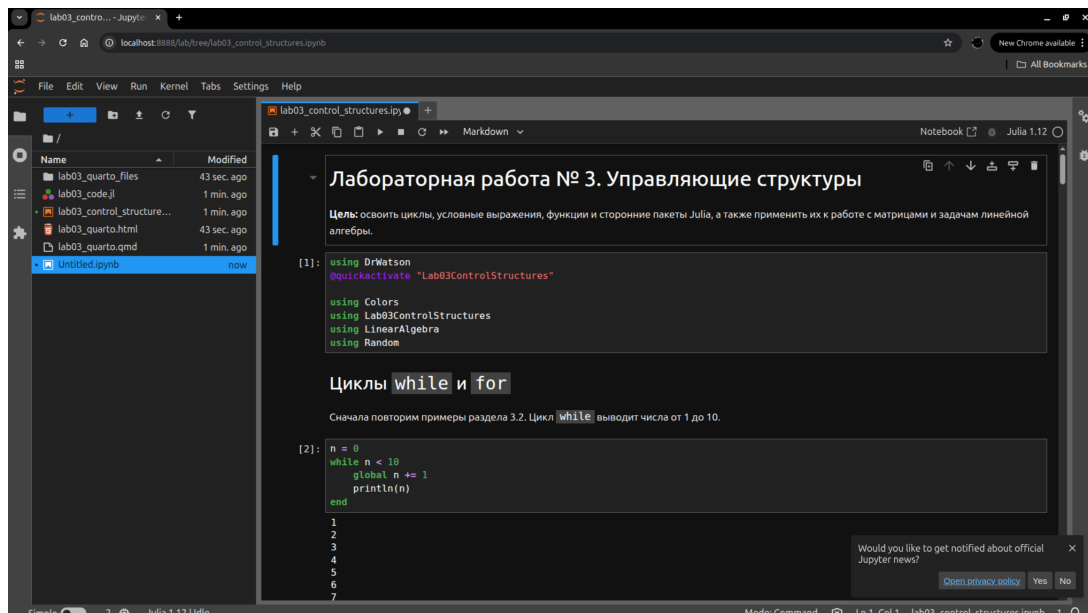


Рисунок 4.23: Начало выполненного Jupyter Notebook

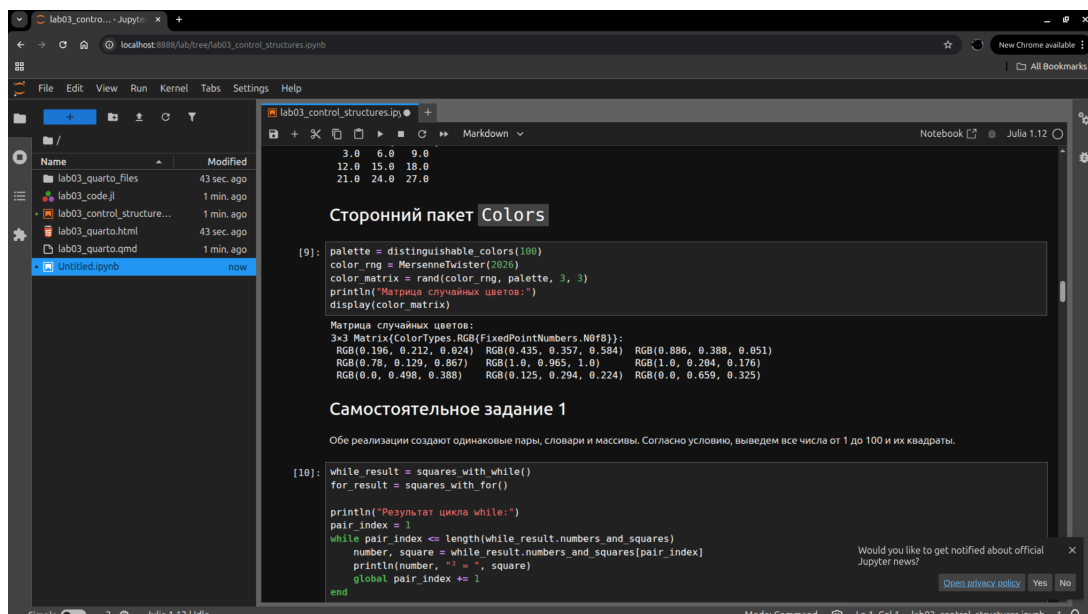


Рисунок 4.24: Матричные задания в Jupyter Notebook

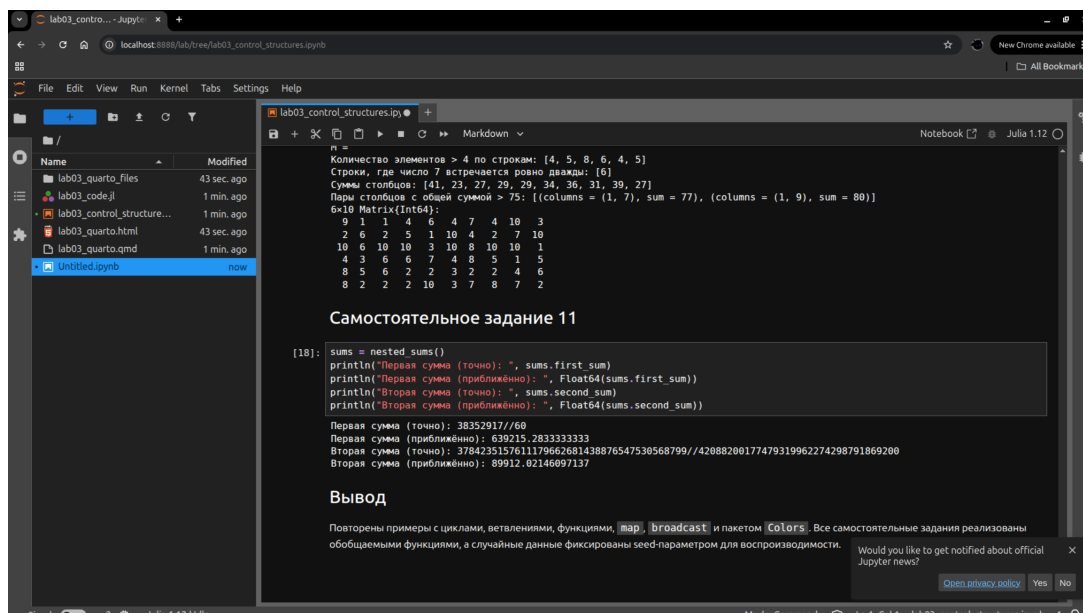


Рисунок 4.25: Заключительная часть Jupyter Notebook

4.13 Автоматические тесты

Тесты охватывают циклы, условия, функции, матричные операции, `outer`, систему уравнений, случайную матрицу, двойные суммы и структуру проекта. Все 54 проверки завершились успешно (Рисунок 4.26).

```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$
[9a3f8284] Random v1.11.0
[ea8e919c] SHA v0.7.0
[9e88b42a] Serialization v1.11.0
[6462fe0b] Sockets v1.11.0
[f409334b] StyledStrings v1.11.0
[fa267f1f] TOML v1.0.3
[a4e569a6] Tar v1.10.0
[8dfed614] Test v1.11.0
[cf7118a7] UUIDs v1.11.0
[4ec0a83e] Unicode v1.11.0
[e66e0078] CompilerSupportLibraries_jll v1.3.0+1
[deac9b47] LibCURL_jll v8.15.0+0
[e37daf67] LibGit2_jll v1.9.0+0
[29816b5a] LibSSH2_jll v1.11.3+1
[14a3606d] MozillaCACerts_jll v2025.11.4
[4536629a] OpenBLAS_jll v0.3.29+0
[450c3c95] OpenSSL_jll v3.5.4+0
[83775a58] Zlib_jll v1.3.1+2
[0e850b90] libblastrampoline_jll v5.15.0+0
[0e850b90] nghttp2_jll v1.64.0+1
[3f19e933] p7zip_jll v17.7.0+0
Info Packages marked with ⚠ have new versions available but compatibility constraints restrict them from upgrading.
Testing Running tests...
Test Summary: | Pass Total Time
Задание 1: циклы | 6 6 0.3s
Test Summary: | Pass Total Time
Задания 2-4: условия и функции | 6 6 0.1s
Test Summary: | Pass Total Time
Задания 5-7: матрицы | 14 14 1.5s
Test Summary: | Pass Total Time
Задание 8: outer | 8 8 0.6s
Test Summary: | Pass Total Time
Задания 9-11 | 11 11 3.4s
Test Summary: | Pass Total Time
Структура и производные материалы | 9 9 0.0s
Testing Lab03ControlStructures tests passed
asvedjmina@veddy-msi:~/university/static_analys/labs/lab03$
```

Рисунок 4.26: Успешное выполнение автоматических тестов

5 Результаты

Основные численные результаты представлены в Таблица 5.1.

Таблица 5.1: Сводные результаты самостоятельной работы

Задание	Проверяемый результат	Значение
1	Последний квадрат	$100^2 = 10000$
4	Размер возрастающей матрицы	3×4
5	Третья степень матрицы	$A^3 = 0$
6	Диагональ $B^T B$	(1500, 1500, 1500)
8	Проверка outer	<code>outer(A,B,*) == A*B</code>
9	Решение системы	[-2, 3, 5, 2, -4]
10	Строка с двумя семёрками	6
11	Первая сумма	38352917/60

В результате сформированы воспроизводимое окружение Julia, модуль с обобщаемыми функциями, literate-источник, выполненный notebook, Quarto-документ, HTML-страница и автоматические тесты.

6 Обсуждение результатов

Все матричные конструкции заданы формулами по индексам, а не перечислением строк, поэтому решения обобщаются на большие размерности. Структурная матрица системы также создаётся по формуле $|i - j| + 1$. Фиксированное начальное состояние генератора делает результаты задания 10 повторяемыми.

Множественная диспетчеризация позволила функции `outer` поддержать два связанных режима: попарную операцию над векторами и обобщённое матричное произведение. Для двойных сумм выбран рациональный тип с `BigInt`; это сохраняет точность и предотвращает переполнение знаменателя.

Единый `literate`-источник исключает расхождение между обычным сценарием, `notebook` и `Quarto`-документом. Автоматические тесты подтверждают численные результаты и наличие производных файлов.

7 Заключение

В ходе лабораторной работы:

1. изучены циклы `while` и `for`, условные выражения и формы определения функций;
2. освоены `map`, `broadcast`, точечный синтаксис и пакет `Colors`;
3. выполнены все одиннадцать самостоятельных заданий;
4. реализованы обобщаемые алгоритмы построения структурных матриц;
5. решена система линейных уравнений и вычислены двойные суммы;
6. создан воспроизводимый проект `DrWatson`;
7. сформированы и проверены `Julia`-, `Jupyter`- и `Quarto`-представления;
8. все 54 автоматические проверки завершились успешно.

Таким образом, цель работы достигнута: управляющие структуры `Julia` освоены и применены к воспроизводимому решению задач линейной алгебры и обработки матриц.

Список литературы

1. *Datseris G., Isensee J., Pech S.* DrWatson.jl Documentation. — URL: <https://juliadynamics.github.io/DrWatson.jl/> (visited on 10/02/2026).
2. *Fredriksson F., Carlsson K.* Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/> (visited on 10/02/2026).
3. *JuliaGraphics.* Colors.jl Documentation. — URL: <https://juliagraphics.github.io/Colors.jl/stable/> (visited on 10/02/2026).
4. *JuliaLang.* IJulia.jl Documentation. — URL: <https://julialang.github.io/IJulia.jl/stable/> (visited on 10/02/2026).
5. *Posit Software, PBC.* Quarto Documentation. — URL: <https://quarto.org/docs/> (visited on 10/02/2026).
6. *The Julia Project.* The Julia Language Documentation. — URL: <https://docs.julialang.org/> (visited on 10/02/2026).

А Полный вычислительный модуль

Ниже приведён полный файл `src/Lab03ControlStructures.jl`. Перенос длинных строк в PDF включён, поэтому листинг остаётся в границах страницы.

```
1 module Lab03ControlStructures
2
3 using LinearAlgebra
4 using Random
5
6 export squares_with_while, squares_with_for,
7       even_or_odd, even_or_odd_ternary, add_one, increasing_matrix,
8       matrix_power_task, matrix_product_task, pattern_matrices,
9       outer, outer_matrices, structured_system, random_matrix_task,
10      nested_sums
11
12 """Получить числа 1:100, их квадраты, словарь и массив квадратов с помощью
13 ↪ `while`."""
14 function squares_with_while(; limit::Integer=100)
15     limit >= 1 || throw(ArgumentError("limit должен быть положительным"))
16     numbers_and_squares = Tuple{Int, Int}[]
17     squares = Dict{Int, Int}()
18     squares_arr = Int[]
19
20     number = 1
```

```

20     while number <= limit
21         square = number^2
22         push!(numbers_and_squares, (number, square))
23         squares[number] = square
24         push!(squares_arr, square)
25         number += 1
26     end
27
28     return (; numbers_and_squares, squares, squares_arr)
29 end
30
31 """Получить числа 1:100, их квадраты, словарь и массив квадратов с помощью `for`."""
32 function squares_with_for(; limit::Integer=100)
33     limit >= 1 || throw(ArgumentError("limit должен быть положительным"))
34     numbers_and_squares = Tuple{Int, Int}[]
35     squares = Dict{Int, Int}()
36     squares_arr = Int[]
37
38     for number in 1:limit
39         square = number^2
40         push!(numbers_and_squares, (number, square))
41         squares[number] = square
42         push!(squares_arr, square)
43     end
44
45     return (; numbers_and_squares, squares, squares_arr)
46 end
47
48 """Вернуть число, если оно чётное, и строку `нечётное` в противном случае."""
49 function even_or_odd(number::Integer)

```

```

50     if iseven(number)
51         return number
52     else
53         return "нечётное"
54     end
55 end
56
57 """Тот же условный выбор, записанный тернарным оператором."""
58 even_or_odd_ternary(number::Integer) = iseven(number) ? number : "нечётное"
59
60 """Увеличить аргумент на единицу."""
61 add_one(value) = value + one(value)
62
63 """Создать матрицу последовательных чисел, применив `add_one` через `map`."""
64 function increasing_matrix(rows::Integer=3, columns::Integer=4)
65     rows > 0 && columns > 0 || throw(ArgumentError("размеры должны быть
↵ положительными"))
66     predecessors = [
67         (row - 1) * columns + column - 1
68         for row in 1:rows, column in 1:columns
69     ]
70     return map(add_one, predecessors)
71 end
72
73 """Вычислить  $A^3$  и заменить третий столбец копии A суммой столбцов 2 и 3."""
74 function matrix_power_task()
75     A = [1 1 3; 5 2 6; -2 -1 -3]
76     A_cubed = A^3
77     A_modified = copy(A)
78     A_modified[:, 3] .= A[:, 2] .+ A[:, 3]

```

```

79     return (; A, A_cubed, A_modified)
80 end
81
82 """Построить B размера `rows × 3` и вычислить C = BTB."""
83 function matrix_product_task(; rows::Integer=15)
84     rows > 0 || throw(ArgumentError("rows должен быть положительным"))
85     B = repeat([10 -10 10], rows, 1)
86     C = transpose(B) * B
87     return (; B, C)
88 end
89
90 """Построить нулевую, единичную и четыре структурные матрицы размера n × n."""
91 function pattern_matrices(; n::Integer=6)
92     n > 0 || throw(ArgumentError("n должен быть положительным"))
93     Z = zeros{Int, n, n}
94     E = ones{Int, n, n}
95     Z1 = copy(Z)
96     Z2 = copy(Z)
97     Z3 = copy(Z)
98     Z4 = copy(Z)
99
100     for row in 1:n, column in 1:n
101         Z1[row, column] = abs(row - column) == 1
102         Z2[row, column] = abs(row - column) in (0, 2)
103         Z3[row, column] = abs(row - (n - column + 1)) in (0, 2)
104         Z4[row, column] = iseven(row + column)
105     end
106
107     return (; Z, E, Z1, Z2, Z3, Z4)
108 end

```

```

109
110 """
111     outer(x, y, operation)
112
113 Применить `operation` ко всем парам элементов векторов `x` и `y`, как `outer()` в R.
114 """
115 function outer(x::AbstractVector, y::AbstractVector, operation::Function)
116     return [operation(left, right) for left in x, right in y]
117 end
118
119 """
120     outer(A, B, operation)
121
122 Обобщить матричное произведение: `operation` заменяет умножение, а результаты по
123 общему индексу складываются. Поэтому `outer(A, B, *) == A * B`.
124 """
125 function outer(A::AbstractMatrix, B::AbstractMatrix, operation::Function)
126     size(A, 2) == size(B, 1) || throw(DimensionMismatch(
127         "число столбцов A должно совпадать с числом строк B",
128     ))
129     common = axes(A, 2)
130     return [sum(operation(A[row, k], B[k, column]) for k in common)
131             for row in axes(A, 1), column in axes(B, 2)]
132 end
133
134 """Создать пять матриц задания 8 посредством `outer`."""
135 function outer_matrices(; n_small::Integer=5, n_large::Integer=10,
136     ↪ n_shift::Integer=9)
137     n_small > 0 && n_large > 0 && n_shift > 0 ||
138         throw(ArgumentError("размеры должны быть положительными"))

```

```

138
139     small = collect(0:(n_small - 1))
140     large = collect(0:(n_large - 1))
141     shifted = collect(0:(n_shift - 1))
142
143     A1 = outer(small, small, +)
144     A2 = outer(small, collect(1:n_small), ^)
145     A3 = outer(small, small, (left, right) -> mod(left + right, n_small))
146     A4 = outer(large, large, (left, right) -> mod(left + right, n_large))
147     A5 = outer(shifted, shifted, (left, right) -> mod(left - right, n_shift))
148
149     return (; A1, A2, A3, A4, A5)
150 end
151
152 """Построить структурную матрицу  $A_{ij}=|i-j|+1$  и решить  $Ax=y$ ."""
153 function structured_system(; y::AbstractVector=[7, -1, -3, 5, 17])
154     n = length(y)
155     n > 0 || throw(ArgumentError("вектор правой части не должен быть пустым"))
156     A = [abs(row - column) + 1 for row in 1:n, column in 1:n]
157     x = A \ y
158     return (; A, y=collect(y), x)
159 end
160
161 """Выполнить три запроса задания 10 для воспроизводимой случайной матрицы."""
162 function random_matrix_task(;
163     seed::Integer=2026,
164     rows::Integer=6,
165     columns::Integer=10,
166     threshold::Integer=4,
167     target::Integer=7,

```

```

168     target_count::Integer=2,
169     pair_threshold::Integer=75,
170 )
171     rows > 0 && columns > 1 || throw(ArgumentError(
172         "требуется положительное число строк и не менее двух столбцов",
173     ))
174     rng = MersenneTwister(seed)
175     M = rand(rng, 1:10, rows, columns)
176
177     greater_counts = vec(sum(M .> threshold; dims=2))
178     matching_rows = findall(vec(sum(M .== target; dims=2)) .== target_count)
179     column_sums = vec(sum(M; dims=1))
180     qualifying_pairs = [
181         (columns=(first_column, second_column),
182         sum=column_sums[first_column] + column_sums[second_column])
183         for first_column in 1:(columns - 1)
184         for second_column in (first_column + 1):columns
185         if column_sums[first_column] + column_sums[second_column] > pair_threshold
186     ]
187
188     return (; M, greater_counts, matching_rows, column_sums, qualifying_pairs)
189 end
190
191 """Вычислить обе двойные суммы задания 11 точно, как рациональные числа."""
192 function nested_sums(; max_i::Integer=20, max_j::Integer=5)
193     max_i > 0 && max_j > 0 || throw(ArgumentError("границы должны быть
194 ↪ положительными"))
195     # BigInt предотвращает переполнение при приведении дробей к общему знаменателю.
196     first_sum = sum(big(i)^4 // big(3 + j) for i in 1:max_i for j in 1:max_j)
197     second_sum = sum(big(i)^4 // big(3 + i * j) for i in 1:max_i for j in 1:max_j)

```

```
197     return (; first_sum, second_sum)
198 end
199
200 end
```

В Полный literate-сценарий

Ниже приведён полный файл `scripts/lab03_literate.jl`, из которого создаются исполняемый сценарий, Jupyter Notebook и Quarto-документ.

```
1  # # Лабораторная работа № 3. Управляющие структуры
2  #
3  # **Цель:** освоить циклы, условные выражения, функции и сторонние пакеты Julia,
4  # а также применить их к работе с матрицами и задачам линейной алгебры.
5
6  using DrWatson
7  @quickactivate "Lab03ControlStructures"
8
9  using Colors
10 using Lab03ControlStructures
11 using LinearAlgebra
12 using Random
13
14 # ## Циклы `while` и `for`
15 #
16 # Сначала повторим примеры раздела 3.2. Цикл `while` выводит числа от 1 до 10.
17
18 n = 0
19 while n < 10
20     global n += 1
```

```

21     println(n)
22 end
23
24 # Переберём массив имён с помощью `while`, а затем с помощью `for`.
25
26 myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
27
28 i = 1
29 while i <= length(myfriends)
30     friend = myfriends[i]
31     println("Hi $friend, it's great to see you!")
32     global i += 1
33 end
34
35 for friend in myfriends
36     println("Hi $friend, it's great to see you!")
37 end
38
39 for number in 1:2:10
40     println(number)
41 end
42
43 # Три эквивалентных способа построения матрицы, элементы которой равны сумме
44 # индексов строки и столбца.
45
46 m, n = 5, 5
47 A_loop = fill{0, (m, n)}
48 for row in 1:m
49     for column in 1:n
50         A_loop[row, column] = row + column

```

```

51     end
52 end
53
54 B_loop = fill(0, (m, n))
55 for row in 1:m, column in 1:n
56     B_loop[row, column] = row + column
57 end
58
59 C_comprehension = [row + column for row in 1:m, column in 1:n]
60 display(A_loop)
61 println("Все три матрицы совпадают: ", A_loop == B_loop == C_comprehension)
62
63 # ## Условные выражения
64
65 function fizzbuzz(number)
66     if number % 3 == 0 && number % 5 == 0
67         return "FizzBuzz"
68     elseif number % 3 == 0
69         return "Fizz"
70     elseif number % 5 == 0
71         return "Buzz"
72     else
73         return number
74     end
75 end
76
77 for number in (3, 5, 15, 17)
78     println("$number → ", fizzbuzz(number))
79 end
80

```

```

81 x, y = 5, 10
82 println("Максимум через тернарный оператор: ", x > y ? x : y)
83
84 # ## Функции
85 #
86 # Функцию можно записать полным, однострочным или анонимным способом.
87
88 function sayhi(name)
89     println("Hi $name, it's great to see you!")
90 end
91
92 function f(value)
93     value^2
94 end
95
96 sayhi2(name) = println("Hi $name, it's great to see you!")
97 f2(value) = value^2
98 sayhi3 = name -> println("Hi $name, it's great to see you!")
99 f3 = value -> value^2
100
101 sayhi("C-3P0")
102 sayhi2("R2-D2")
103 sayhi3("BB-8")
104 println("Три способа задать квадрат: ", (f(42), f2(42), f3(42)))
105
106 # `sort` возвращает копию, а `sort!` изменяет исходный массив.
107
108 v = [3, 5, 2]
109 println("sort(v) = ", sort(v), "; исходный v = ", v)
110 sort!(v)

```

```

111 println("После sort!(v): ", v)
112
113 # `map` и `broadcast` применяют функцию поэлементно.
114
115 println("map(f, [1, 2, 3]) = ", map(f, [1, 2, 3]))
116 println("map(x -> x^3, [1, 2, 3]) = ", map(value -> value^3, [1, 2, 3]))
117 println("broadcast(f, [1, 2, 3]) = ", broadcast(f, [1, 2, 3]))
118
119 A_broadcast = [column + 3row for row in 0:2, column in 1:3]
120 println("A =")
121 display(A_broadcast)
122 println("f(A) = A^2 =")
123 display(f(A_broadcast))
124 println("f.(A) – квадраты элементов =")
125 display(f.(A_broadcast))
126
127 expression_1 = A_broadcast .+ 2 .* f.(A_broadcast) ./ A_broadcast
128 expression_2 = @. A_broadcast + 2 * f(A_broadcast) / A_broadcast
129 expression_3 = broadcast(value -> value + 2 * f(value) / value, A_broadcast)
130 display(expression_1)
131 println("Три формы broadcast совпадают: ", expression_1 == expression_2 ==
    ↪ expression_3)
132
133 # ## Сторонний пакет `Colors`
134
135 palette = distinguishable_colors(100)
136 color_rng = MersenneTwister(2026)
137 color_matrix = rand(color_rng, palette, 3, 3)
138 println("Матрица случайных цветов:")
139 display(color_matrix)

```

```

140
141 # ## Самостоятельное задание 1
142 #
143 # Обе реализации создают одинаковые пары, словари и массивы. Согласно условию,
144 # выведем все числа от 1 до 100 и их квадраты.
145
146 while_result = squares_with_while()
147 for_result = squares_with_for()
148
149 println("Результат цикла while:")
150 pair_index = 1
151 while pair_index <= length(while_result.numbers_and_squares)
152     number, square = while_result.numbers_and_squares[pair_index]
153     println(number, "² = ", square)
154     global pair_index += 1
155 end
156
157 println("Результат цикла for:")
158 for (number, square) in for_result.numbers_and_squares
159     println(number, "² = ", square)
160 end
161
162 println("squares[10] = ", for_result.squares[10])
163 println("squares_arr = ", for_result.squares_arr)
164 println("Результаты while и for совпадают: ", while_result == for_result)
165
166 # ## Самостоятельные задания 2–4
167
168 for number in 1:4
169     println("if: $number → ", even_or_odd(number),

```

```

170         "; тернарный: ", even_or_odd_ternary(number))
171     end
172
173     println("add_one(41) = ", add_one(41))
174     A_increasing = increasing_matrix(3, 4)
175     println("Матрица последовательных чисел, построенная через map(add_one, ...):")
176     display(A_increasing)
177
178     # ## Самостоятельное задание 5
179
180     power_task = matrix_power_task()
181     println("A =")
182     display(power_task.A)
183     println("A³ =")
184     display(power_task.A_cubed)
185     println("A после замены третьего столбца:")
186     display(power_task.A_modified)
187
188     # ## Самостоятельное задание 6
189
190     product_task = matrix_product_task()
191     println("B =")
192     display(product_task.B)
193     println("C = BᵀB =")
194     display(product_task.C)
195
196     # ## Самостоятельное задание 7
197
198     patterns = pattern_matrices()
199     for name in (:Z, :E, :Z1, :Z2, :Z3, :Z4)

```

```

200     println(name, " =")
201     display(getproperty(patterns, name))
202 end
203
204 # ## Самостоятельное задание 8
205 #
206 # Для векторов функция `outer` применяет операцию ко всем парам элементов. Для
207 # матриц она сворачивает результат по общему индексу, поэтому при операции `*`
208 # совпадает с обычным матричным произведением.
209
210 left_matrix = [1 2; 3 4]
211 right_matrix = [5 6; 7 8]
212 println("outer(A, B, *) == A * B: ",
213         outer(left_matrix, right_matrix, *) == left_matrix * right_matrix)
214
215 outer_result = outer_matrices()
216 for name in (:A1, :A2, :A3, :A4, :A5)
217     println(name, " =")
218     display(getproperty(outer_result, name))
219 end
220
221
222 # ## Самостоятельное задание 9
223
224 system_result = structured_system()
225 println("Структурная матрица системы A =")
226 display(system_result.A)
227 println("Решение x = ", system_result.x)
228 println("Проверка A*x ≈ y: ", system_result.A * system_result.x ≈ system_result.y)
229

```

```

230 # ## Самостоятельное задание 10
231
232 random_result = random_matrix_task()
233 println("M =")
234 display(random_result.M)
235 println("Количество элементов > 4 по строкам: ", random_result.greater_counts)
236 println("Строки, где число 7 встречается ровно дважды: ",
    ↪ random_result.matching_rows)
237 println("Суммы столбцов: ", random_result.column_sums)
238 println("Пары столбцов с общей суммой > 75: ", random_result.qualifying_pairs)
239
240 # ## Самостоятельное задание 11
241
242 sums = nested_sums()
243 println("Первая сумма (точно): ", sums.first_sum)
244 println("Первая сумма (приближённо): ", Float64(sums.first_sum))
245 println("Вторая сумма (точно): ", sums.second_sum)
246 println("Вторая сумма (приближённо): ", Float64(sums.second_sum))
247
248 # ## Вывод
249 #
250 # Повторены примеры с циклами, ветвлениями, функциями, `map`, `broadcast` и
251 # пакетом `Colors`. Все самостоятельные задания реализованы обобщаемыми
252 # функциями, а случайные данные фиксированы seed-параметром для воспроизводимости.

```